National University of Computer and Emerging Sciences

Object Oriented Programming Lab (CL1004)

Date: 06/05/2025

Course Instructor(s)

Mr. Muhammed Monis, Mr. Abullah Shaikh, Ms. Almas Ayesha Ansari, Ms. Alishba Subhani

Lab Final Exam (A)

Total Time: 120 minutes
Total Marks: 50
Total Questions: **03**

Semester: SP-2025
Campus: Karachi
Dept: Computer Science

Comment is your best friend

LLO 03:Demonstrate advanced C++ OOP by implementing inheritance with access modifiers and virtual inheritance; compile-time polymorphism through function and operator overloading; run-time polymorphism via virtual functions, overriding, abstract classes, and pure virtual functions; and utilizing friend functions and classes for controlled flow.

Q1. [15 marks]

The Academic Affairs department at a university needs a system to manage **staff members** involved in teaching and academic coordination. Employees are categorized into **Lecturers**, **Course Coordinators**, and **Dual Role Staff** (who serve as both). Performance is tracked through teaching effectiveness and course management metrics. Sensitive data must remain protected, with external modules only allowed to read but not modify the data.

Requirements:

Base Staff Class – 2.5 points

Data Members:
staffID (string): Unique identifier , name (string): Full name of the staff member

Function:
displayInfo() – virtual: Displays ID and name

Lecturer Class – 2.5 points

- **Additional Members**:
  - teachingHours (int)
  - studentFeedback (float; 0–5 scale)
- **Functionality**:
  - Overrides displayInfo() to show lecturer metrics
  - Friend function to update teachingHours and studentFeedback (with validation)

CourseCoordinator Class – 2.5 points

- **Additional Members**:
  - coursesManaged (int; must be ≥ 1)
  - curriculumRating (float; 0–10 scale)
- **Functionality**:
  - Overrides displayInfo() to show coordinator metrics
  - Friend function to adjust coursesManaged and curriculumRating (with validation)

DualRoleStaff Class – 2.5 points

Inherits from both Lecturer and CourseCoordinator
Overrides displayInfo() to show **all metrics** and a **combined performance score**:
Score = studentFeedback * 2 + curriculumRating
Ensures there is only **one instance** of staffID and name

AcademicAudit Class – 2.5 points

Can inspect internal performance metrics of Lecturers and Coordinators (via friendship)
Cannot modify metrics
Ensures encapsulation of sensitive academic performance data

Constraints – 2.5 points

- No public access to studentFeedback or curriculumRating
- Avoid duplication of ID/name in DualRoleStaff
- Proper validation and clean class hierarchy

Sample Input:

```
Lecturer lec1("L001", "Dr. Ayesha", 20, 4.5f);
CourseCoordinator cc1("C002", "Dr. Raza", 3, 9.0f);
DualRoleStaff drs("D003", "Dr. Kamran", 25, 4.8f, 2, 8.5f);
lec1.displayInfo();
cc1.displayInfo();
drs.displayInfo();
adjustLecturer(lec1, 24, 4.9f);
adjustCoordinator(cc1, 4, 9.5f);
std::cout << "\nAfter Adjustment:\n";
lec1.displayInfo();
cc1.displayInfo();
AcademicAudit audit;
audit.inspectLecturer(lec1);
audit.inspectCoordinator(cc1);
```

Sample Output:

```
Lecturer Info:
ID: L001, Name: Dr. Ayesha
Teaching Hours: 20
Student Feedback: 4.5

Course Coordinator Info:
ID: C002, Name: Dr. Raza
Courses Managed: 3
Curriculum Rating: 9.0

DualRoleStaff Info:
ID: D003, Name: Dr. Kamran
Teaching Hours: 25
Student Feedback: 4.8
Courses Managed: 2
Curriculum Rating: 8.5
Combined Performance Score: 18.1

After Adjustment:
Lecturer Info:
ID: L001, Name: Dr. Ayesha
Teaching Hours: 24
Student Feedback: 4.9

Course Coordinator Info:
ID: C002, Name: Dr. Raza
Courses Managed: 4
Curriculum Rating: 9.5

Academic Audit Access:
Lecturer Metrics – Hours: 24, Feedback: 4.9
Coordinator Metrics – Courses: 4, Curriculum Rating: 9.5
```

LLO 03:Demonstrate advanced C++ OOP by implementing inheritance with access modifiers and virtual inheritance; compile-time polymorphism through function and operator overloading; run-time polymorphism via virtual functions, overriding, abstract classes, and pure virtual functions; and utilizing friend functions and classes for controlled flow.

Q2. [15 marks]

You are building a system to monitor **Smart Home Appliances**. The program should track device models, their energy consumption, and any advanced features. Newer devices come with additional properties and behaviors.

Base Requirements for All Devices - 3 points:

Every device must store:

model (string): The device model name (e.g., "SmartFridge X100")
energyConsumption (int): Initialized to 10 kWh when created

All devices must implement a core function runCycle() that:

Prints an activity specific to the device type
Increases energy consumption by a fixed amount per cycle
Caps energy at 100 kWh
A status display method showing the model and energy used

Specialized Device Types - 5 points:

SmartFridge Series - 1.5 points:

Activity: "Cooling groceries"
Energy increase: +15 kWh per cycle

SmartWasher Series - 1.5 points:

Activity: "Running laundry cycle"
Energy increase: +30 kWh per cycle

SmartOven Series - 2 points:

Additional property: temperatureSetting (starts at 180°C)
Activity: "Baking at [temperatureSetting]°C"
Energy increase: +20 kWh per cycle
Special functionality: Overload ++ to increase temperatureSetting by 20°C
Status display must show both energy and current temperature setting

• **System-wide Features - 7 points:**

Compare devices using > to determine which consumed more energy
Print device status using << in the format:
[Model]: [Energy] kWh (or ...kWh, Oven Temp: [temp]°C for SmartOven)
Demonstrate polymorphism using base class pointers or references

Restrictions: No multiple inheritance, No global hardcoding , Minimize duplicate code, Use clean inheritance and operator overloading principles

Sample Input:

```
SmartDevice* devices[3];
devices[0] = new SmartFridge("SmartFridge X100");
devices[1] = new SmartWasher("SmartWasher Turbo");
devices[2] = new SmartOven("SmartOven Pro");
for (int i = 0; i < 2; ++i)
    devices[i]->runCycle();
++(*devices[2]); // increase oven temperature
devices[2]->runCycle();
for (int i = 0; i < 3; ++i)
    std::cout << *devices[i] << std::endl;
if (*devices[1] > *devices[0])
     std::cout << devices[1]->getModel() << " is more energy-intensive than " << devices[0]->getModel();
```

Sample Output:

SmartFridge X100 is Cooling groceries... SmartFridge X100 is Cooling groceries...
SmartWasher Turbo is Running laundry cycle... SmartWasher Turbo is Running laundry cycle...
SmartOven Pro is Baking at 200°C... SmartFridge X100: 40 kWh
SmartWasher Turbo: 70 kWh
SmartOven Pro: 30 kWh, Oven Temp: 200°C
SmartWasher Turbo is more energy-intensive than SmartFridge X100

LLO 04: Demonstrate proficiency in C++ generic programming and robust error management by implementing function and class templates, handling exceptions, and performing practical file operations and IOStream-based input/output.

Q3 [20 marks]

Ali is building a Smart Fleet Management System for an autonomous transportation company. The system is responsible for managing different types of vehicles, including delivery vans, passenger cars, and autonomous drones. Each vehicle has a unique identifier, a top speed, and an energy level (e.g., fuel or battery percentage). Some types of vehicles also require special operating permissions (e.g., for drones or heavy vehicles), while others can be used without any restrictions.

To ensure the system is scalable and easy to manage, Ali plans to design a solution that works efficiently for different types of vehicles, ensures secure operation through permission validation, and provides mechanisms for saving/loading fleet information to and from files.

Mentioned Below are the requirements

1. Define a general-purpose base class for vehicles *(2 points)*

Create a base class to represent vehicles with a function to return the vehicle ID, A function to return the maximum speed of the vehicle and a function to return the energy level (e.g., fuel or battery).Ensure that specific vehicle types (like delivery vans, passenger cars, and drones) provide their own implementations of these behaviors.

2. Implement multiple vehicle types *(2 points)*

Create classes for DeliveryVan (include additional attributes like cargo capacity), PassengerCar (include number of seat and AutonomousDrone (include maximum flight altitude) Each type must provide relevant data and behavior.

3. Add a mechanism for permission validation *(2 points)*

Implement a separate class that holds a list of permission codes required to operate certain vehicles (e.g., heavy-duty or airspace licenses). This class should have a function to check if a user's list of credentials includes all required permissions.

4. Introduce extended vehicle types *(2 points)*

Create additional vehicle types such as ElectricDeliveryVan (tracks battery level instead of fuel), HybridPassengerCar (has a mode switch between electric and fuel). These types should override behavior where necessary and add relevant member variables.

5. Develop a manager class to store and operate on a vehicle collection *(3 points)*

Create a manager class that should Add new vehicle objects to the fleet, Display details of all vehicles, Calculate and return the average energy level across all vehicles. Verifying if a user is allowed to operate a particular vehicle based on their credentials.

6. Integrate file input/output and exception handling *(4 points)*

Extend your manager class to support saving the current fleet to a text or binary file, Loading a fleet from a file and reconstructing the state, Handling potential issues such as missing files, malformed data, or duplicate vehicle IDs using a custom exception class.

7. Demonstrate system usage (*5 points)*

In your main function instantiate and add different vehicle types to the manager class.

1.Save the fleet to a file, then load it back.

2.Display all vehicle details.

3.Demonstrate a case where a user is allowed and another case where a user is denied access to operate a vehicle.

4.Catch and handle any exceptions that occur during file loading or eligibility checks.

Sample Input:
Fleet.txt:
Format: VehicleType,ID,Speed,Energy,[ExtraData...]
DeliveryVan,VN101,120,40,Cargo:500
PassengerCar,PC202,180,70,Seats:5
AutonomousDrone,DR303,90,85,Altitude:500
ElectricDeliveryVan,EV404,110,88,Battery:88
HybridPassengerCar,HC505,160,50,Mode:Hybrid
Console Input/output:
Display the content of the file at the start of the program

Sample Output:
//When checking user eligibility
Enter user ID: USER001
Enter user credentials (comma-separated): DRONE_CERT,HEAVY_VEHICLE
Eligibility Check:
- Can operate DR303 (AutonomousDrone): Yes
- Can operate VN101 (DeliveryVan): Yes
- Can operate EV404 (ElectricDeliveryVan): Yes
- Can operate HC505 (HybridPassengerCar): Yes
Enter user ID: USER002
Enter user credentials (comma-separated): NONE
Eligibility Check:
- Can operate DR303 (AutonomousDrone): No - Missing: DRONE_CERT
- Can operate VN101 (DeliveryVan): Yes
- Can operate EV404 (ElectricDeliveryVan): Yes
- Can operate HC505 (HybridPassengerCar): Yes
When Displaying Average Energy Level:
Average Energy Level Across Fleet: 66.6%
On Exception (e.g., malformed file or duplicate ID):
Loading fleet from file: fleet_corrupt.txt
Error: FleetParseException - Duplicate Vehicle ID 'PC202' found.